

Geometry Operations with Raster Data and Data Cubes

Alexander Abuabara

DAEN 489 ISEN 489 Spring 2026



ENGINEERING
TEXAS A&M UNIVERSITY

Overview

- Geometric raster operations include the shifting, flipping, mirroring, scaling, rotation or warping of images
 - These operations are necessary for a variety of applications including georeferencing (used to allow images to be overlaid on an accurate map with a known CRS)
- A variety of georeferencing techniques exist, including:
 - Georectification based on known ground control points
 - Orthorectification, which also accounts for local topography
 - Image registration is used to combine images of the same thing, but shot from different sensors by aligning one image with another (in terms of coordinate system and resolution)
- R is rather unsuitable for the first two points since these often require manual intervention which is why they are usually done with the help of dedicated GIS software
 - On the other hand, aligning several images is possible

Geometric Operations on Raster Data

Resources:

- R <https://r.geocompx.org/geometry-operations#geo-ras>
- Python <https://py.geocompx.org/04-geometry-operations#sec-geo-ras>

Geometric Intersections

- To retrieve a spatial output, we can use almost the same subsetting syntax.
- The only difference is that we must make clear that we would like to keep the matrix structure by setting the drop argument to **FALSE**.
- This will return a raster object containing the cells whose midpoints overlap with **clip**.
- For the same operation, we can also use the **intersect()** and **crop()** command.

```
library(sf)
library(dplyr)
library(spData)
library(terra)

elev = rast(system.file("raster/elev.tif",
  package = "spData"))

clip = rast(xmin = 0.9,
  xmax = 1.8,
  ymin = -0.45,
  ymax = 0.45,
  resolution = 0.3,
  vals = rep(1, 9))

elev[clip, drop = FALSE]
```

Extent and Origin

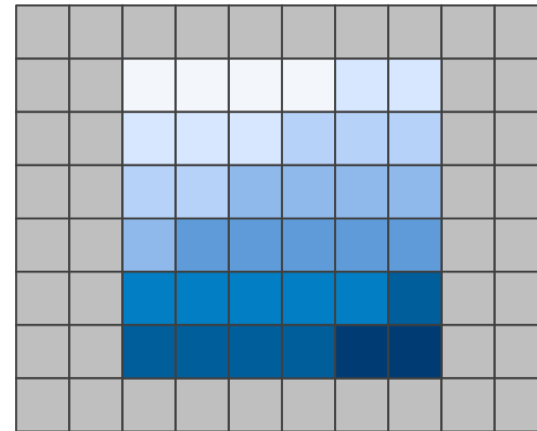
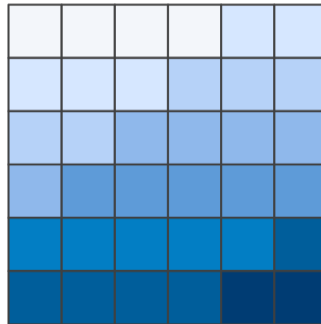
- When merging or performing map algebra on rasters, their resolution, projection, origin and/or extent must match.
- Otherwise, how should we add the values of one raster with a resolution of 0.2 decimal degrees to a second raster with a resolution of 1 decimal degree?
- The same problem arises when we would like to merge satellite imagery from different sensors with different projections and resolutions.
- We can deal with such mismatches by aligning the rasters.

```
elev = rast(system.file("raster/elev.tif",  
  package = "spData"))
```

```
elev_2 = extend(elev, c(1, 2))
```

```
elev_3 = elev + elev_2
```

```
# Error: [+] extents do not match
```



Original raster (left) and the same raster (right) extended by one row on the top and bottom and two columns on the left and right.

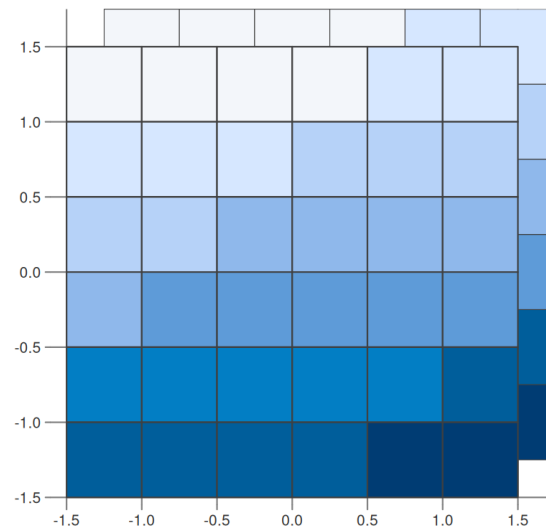
Extent and Origin

- However, we can align the extent of two rasters with **extend()**.
 - Instead of telling the function how many rows or columns should be added (as done before), we allow it to figure it out by using another raster object.
- Here, we extend the **elev** object to the extent of **elev_2**.
 - The values of the newly added rows and columns are set to **NA**.
- The origin of a raster is the cell corner closest to the coordinates (0, 0).
 - The **origin()** function returns the coordinates of the origin.
- If two rasters have different origins, their cells do not overlap completely which would make map algebra impossible.

```
elev_4 = extend(elev, elev_2)

origin(elev_4)
# [1] 0 0

# change the origin
origin(elev_4) = c(0.25, 0.25)
```



Rasters with identical values but different origins.

Aggregation and Disaggregation

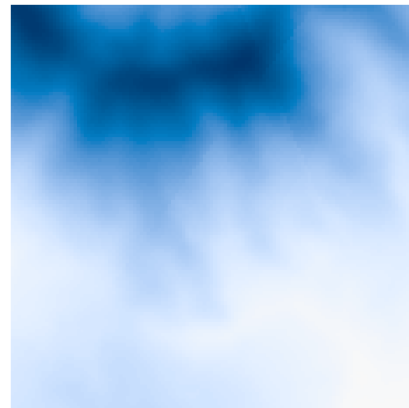
- Raster datasets can also differ regarding their resolution.
- To match resolutions, one can either decrease (`aggregate()`) or increase (`disagg()`) the resolution of one raster.
- As an example, we here change the spatial resolution of dem by a factor of 5.
- The output cell value is going to correspond to the mean of the input cells (could use other functions as well: `median()`, `sum()`, etc.).

```
# install.packages("devtools")
# devtools::install_github("geocompx/geocompkg")

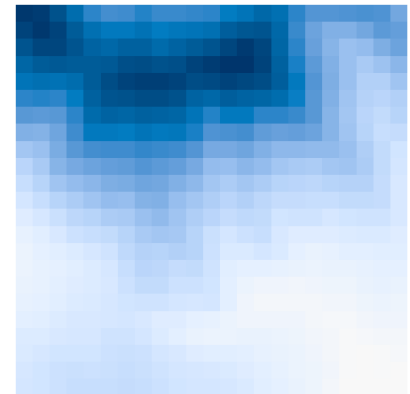
dem = rast(system.file("raster/dem.tif",
  package = "spDataLarge"))

dem_agg = aggregate(dem, fact = 5, fun = mean)
```

A. Original



B. Aggregated

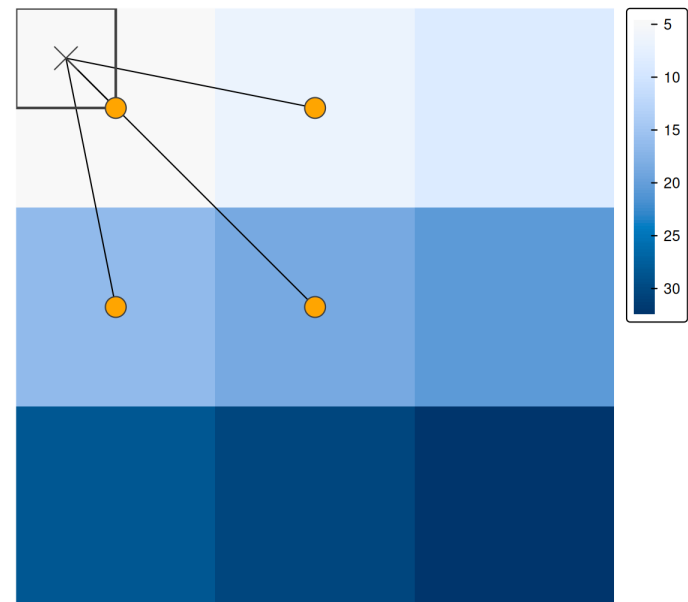


Original raster (left) and aggregated raster (right).

Aggregation and Disaggregation

- The **disagg()** function increases the resolution of raster objects.
- It comes with two methods on how to compute the values of the newly created cells: the default method (**method = "near"**) simply gives all output cells the value of the input cell, and hence duplicates values, which translates into a 'blocky' output.
- The bilinear method uses the four nearest pixel centers of the input image to compute an average weighted by distance.

```
dem_disagg = disagg(dem_agg, fact = 5, method =  
  "bilinear")  
  
identical(dem, dem_disagg)  
  
# [1] FALSE
```

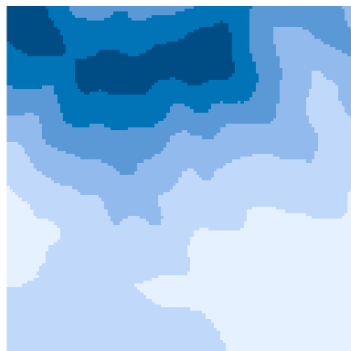


The distance-weighted average of the four closest input cells determine the output when using the bilinear method for disaggregation.

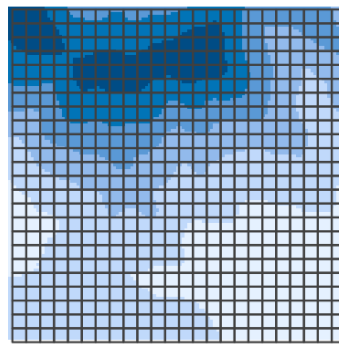
Resampling

- The methods of aggregation and disaggregation are only suitable when we want to change the resolution of our raster by the aggregation/disaggregation factor.
- However, what to do when we have two or more rasters with different resolutions and origins?
- This is the role of resampling: a process of computing values for new pixel locations.

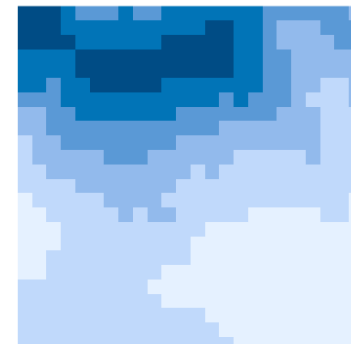
Original raster



Target raster



Resampled raster



Resampling of an original (input) raster into a target raster with custom resolution and origin.

Resampling methods for estimating values for a raster with different resolutions/origins

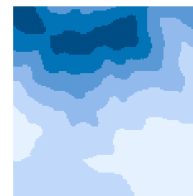
- **Nearest neighbor:** assigns the value of the nearest cell of the original raster to the cell of the target one. This is a fast simple technique that is usually suitable for resampling categorical rasters.
- **Bilinear interpolation:** assigns a weighted average of the four nearest cells from the original raster to the cell of the target one. This is the fastest method that is appropriate for continuous rasters.
- **Cubic interpolation:** uses values of the 16 nearest cells of the original raster to determine the output cell value, applying third-order polynomial functions. Used for continuous rasters and results in a smoother surface compared to bilinear interpolation but is computationally more demanding.
- **Cubic spline interpolation:** also uses values of the 16 nearest cells of the original raster to determine the output cell value but applies cubic splines. Used for continuous rasters.
- **Lanczos windowed sinc-resampling:** uses values of the 36 nearest cells of the original raster to determine the output cell value. Used for continuous rasters.

Resampling Example

- To apply resampling, the terra package provides a **resample()** function.
- It accepts an input raster (x), a raster with target spatial properties (y), and a resampling method (method).
- We need a raster with target spatial properties to see how the resample() function works.
- For this example, we create target_rast, but you would often use an already existing raster object.

```
target_rast = rast(xmin = 794650,  
  xmax = 798250,  
  ymin = 8931750,  
  ymax = 8935350,  
  resolution = 300,  
  crs = "EPSG:32717")  
  
dem_resampl = resample(dem,  
  y = target_rast,  
  method = "bilinear")
```

Original raster



near



bilinear



cubic



cubicspline



lanczos

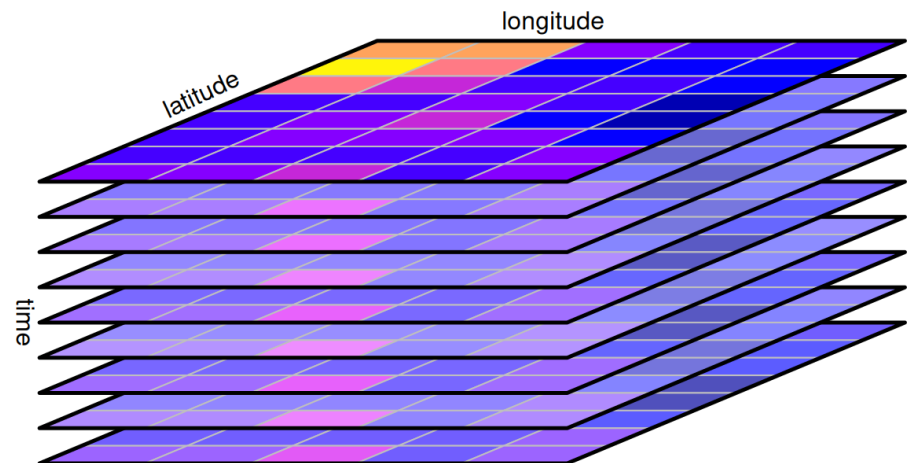


Visual comparison of the original raster and five different resampling methods.

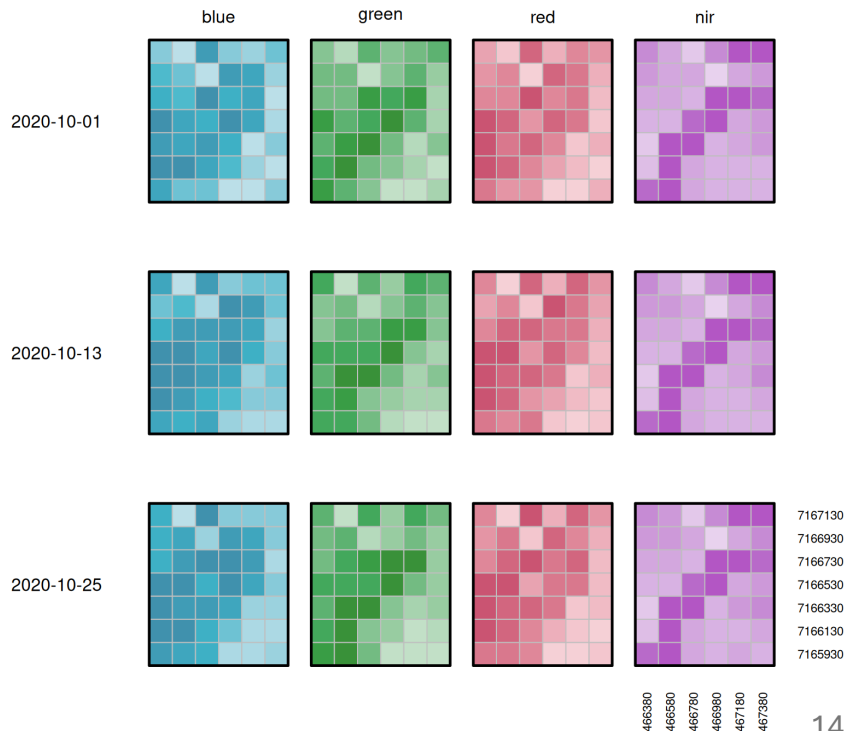
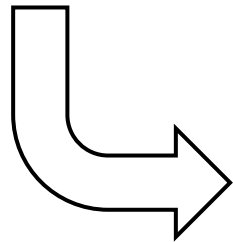
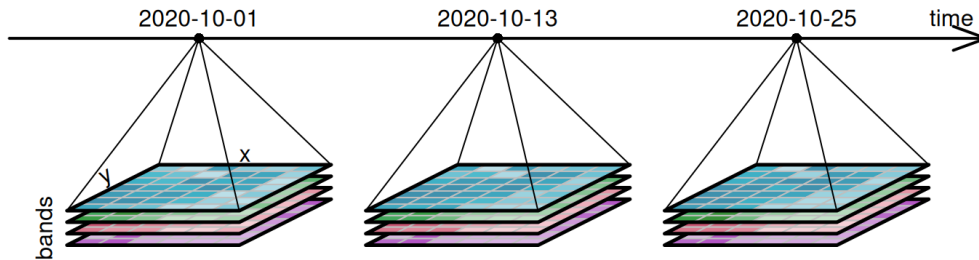
Data Cubes

- Data cubes arise naturally when we observe properties of a set of geometries repeatedly over time.
 - Time information may sometimes be considered as an attribute of a feature, such as when we register the year of construction of a building or the date of birth of a person.
 - One way of considering our world is that of a four-dimensional space, with three space dimensions and one time dimension.
- We define data cubes as array data with one or more array dimensions associated with space and/or time.
- A canonical form of a data cube is:
- Example in R:

```
library(terra)  
r1 <- rast("raster1.tif")  
r2 <- rast("raster2.tif")  
cube <- c(r1, r2)
```

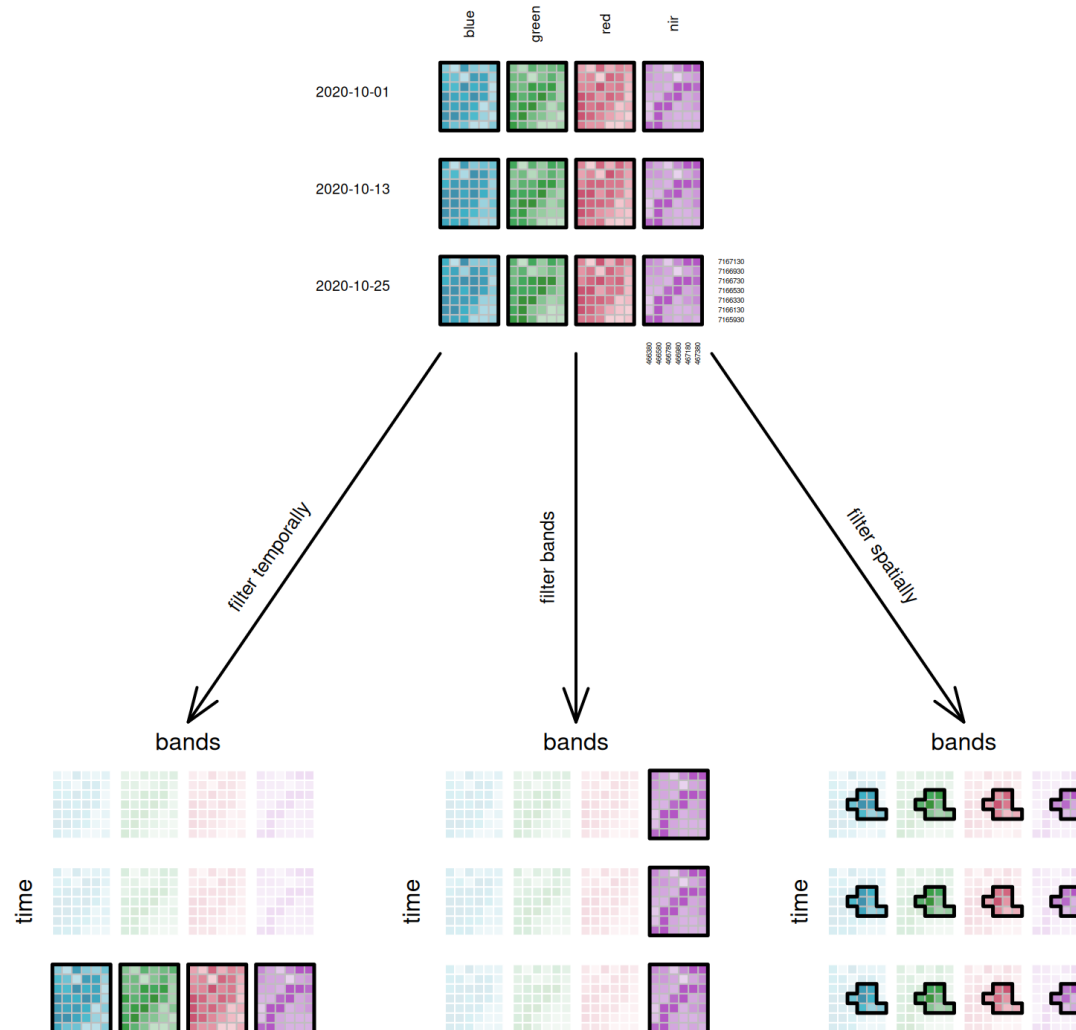


A Four-Dimensional Data Cube

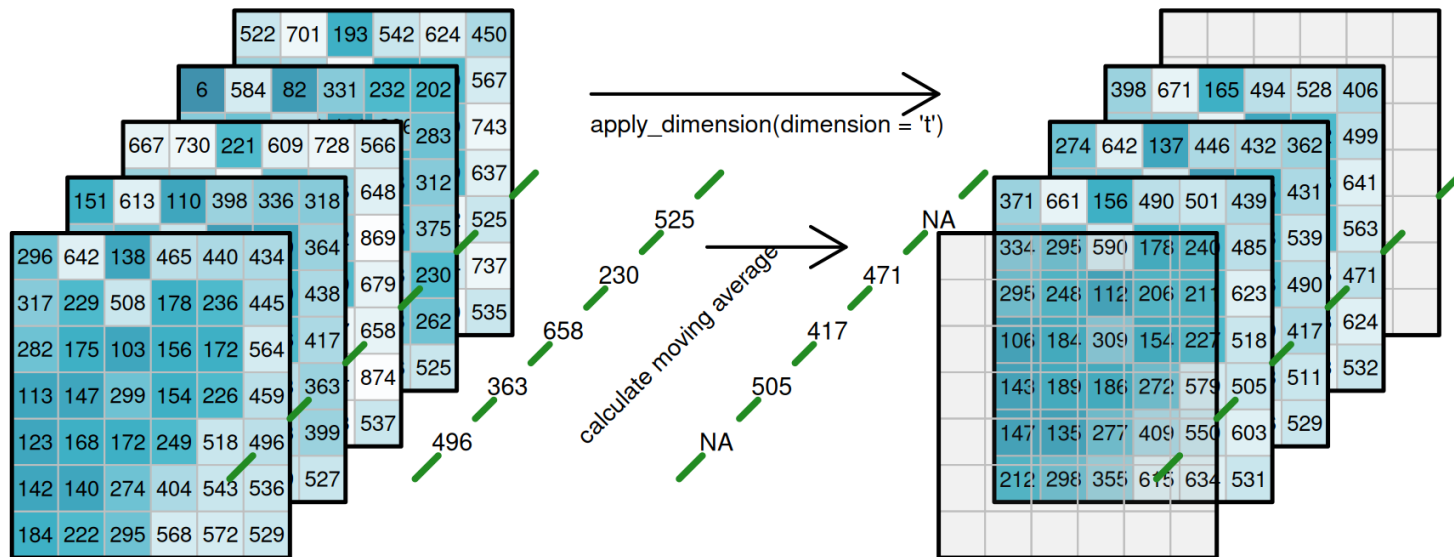


Operations on Data Cubes

Slicing a cube: Filter

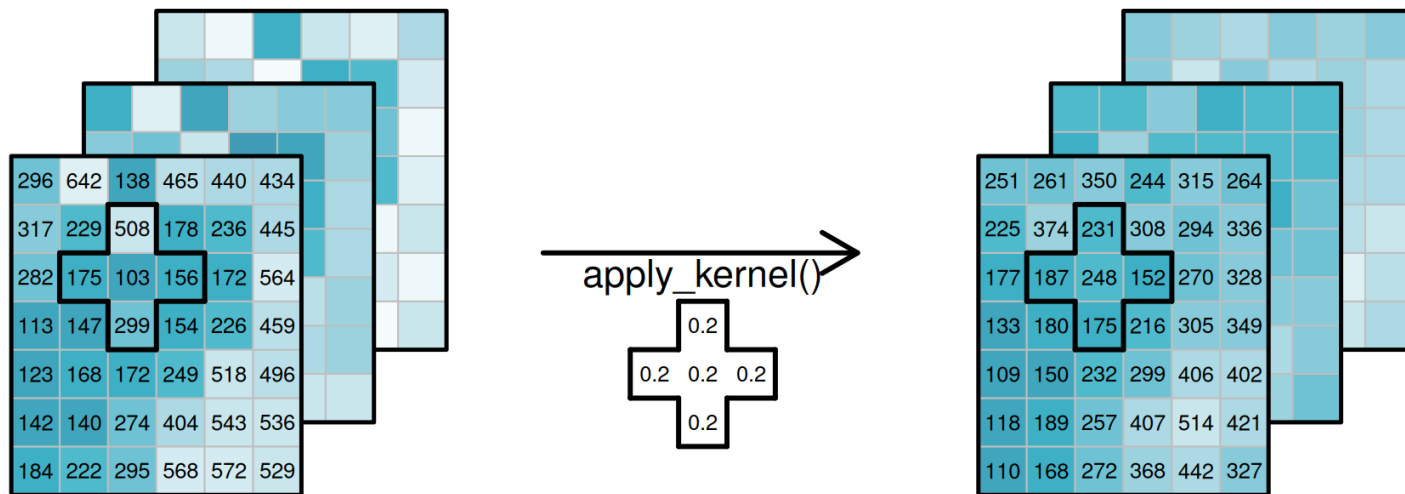


Applying Functions to Dimensions



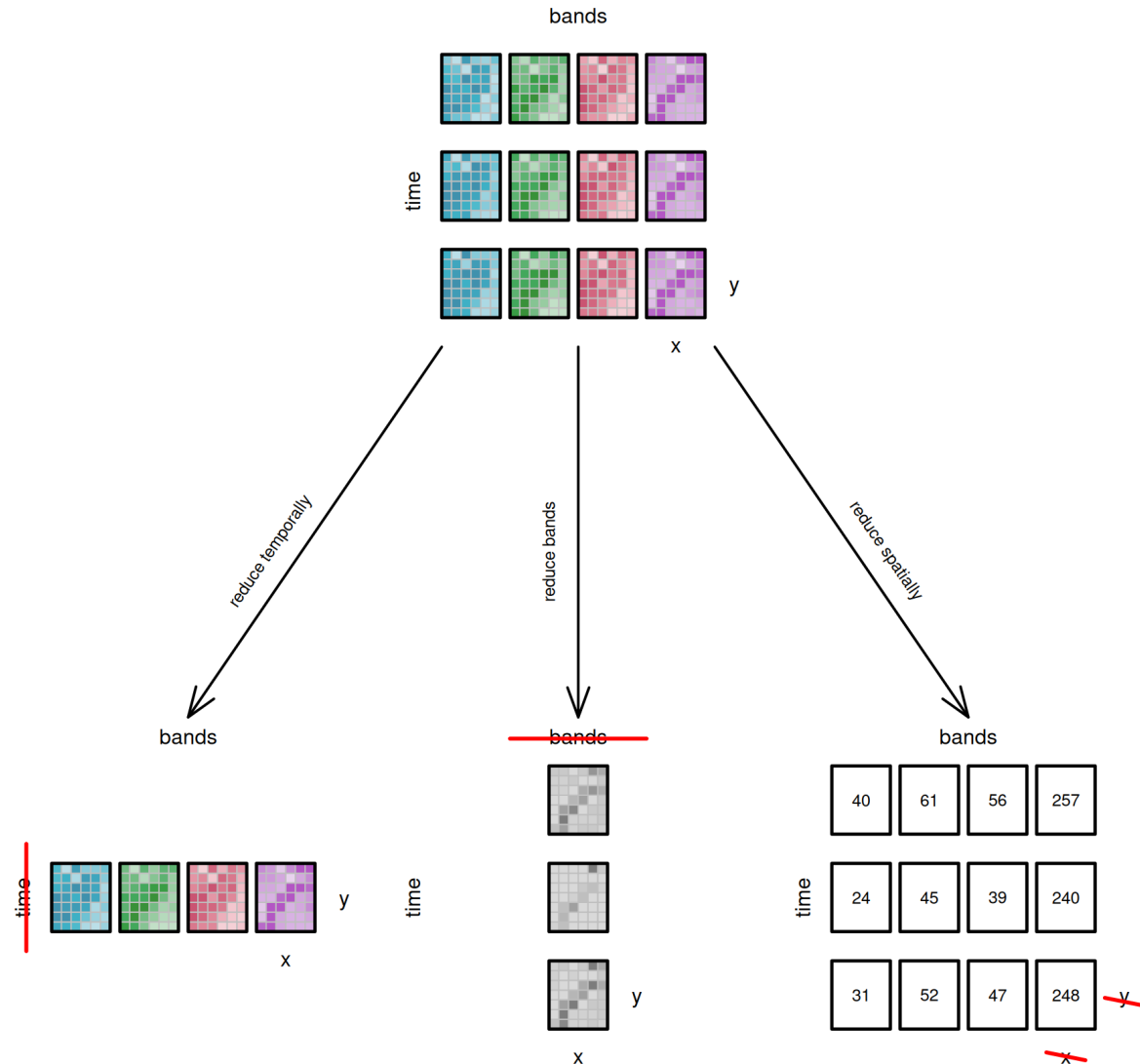
Low-pass filtering of time series.

Applying Functions to Dimensions

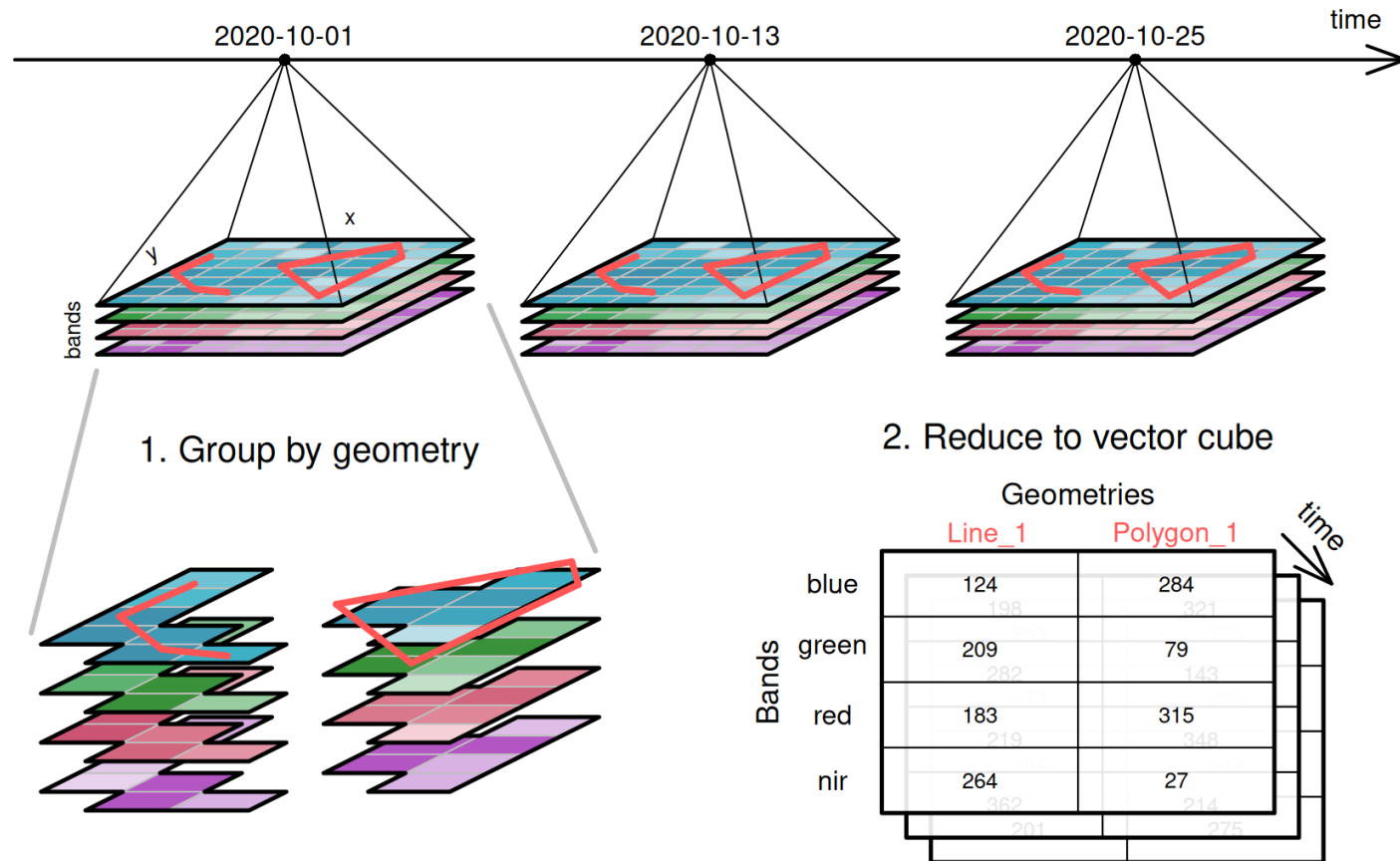


Low pass filtering of spatial slices.

Reducing Dimensions

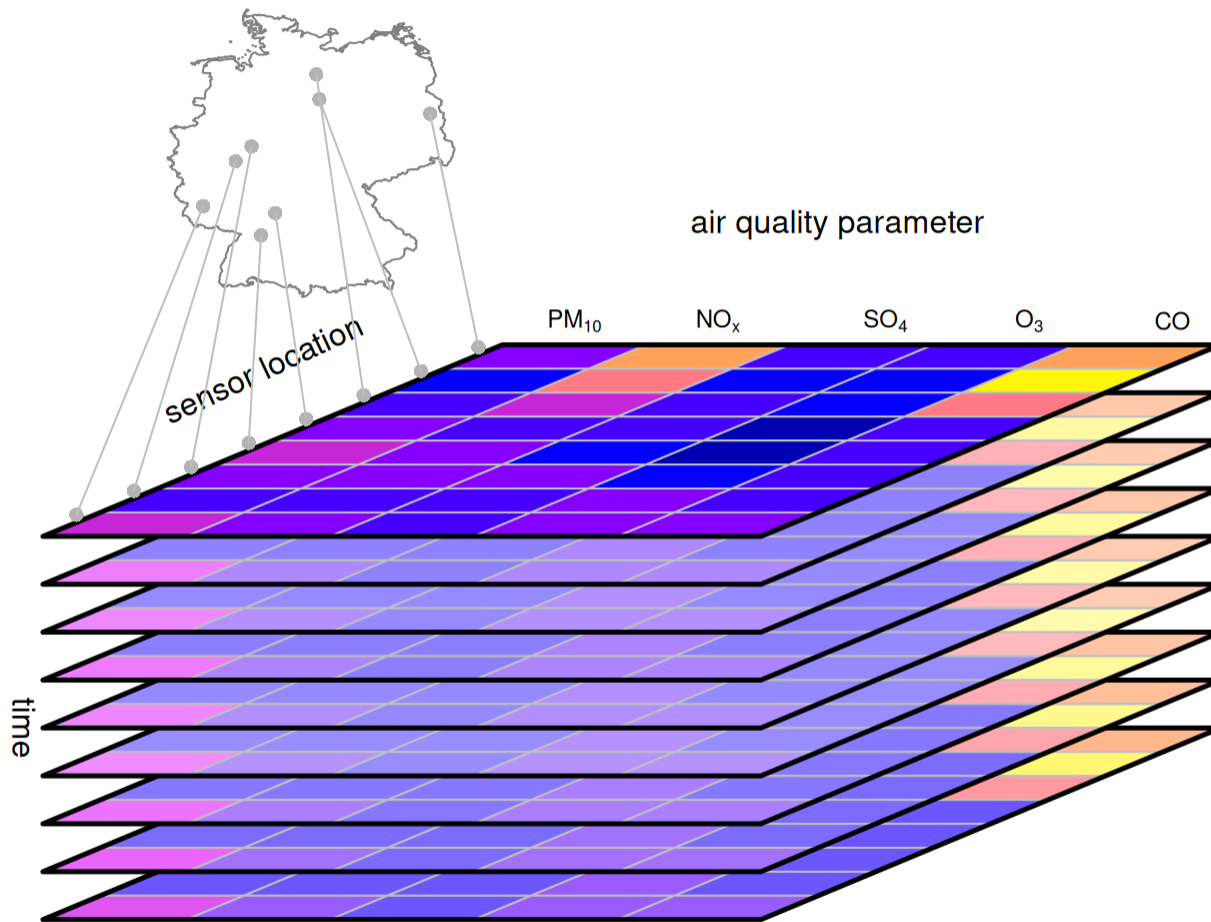


Aggregating Raster to Vector Cubes



Example:

Vector data cube with air quality time series



Tips

```
# before stacking, make sure:
compareGeom(r1, r2)

# they must have: same CRS, same resolution, same extent, same origin
# if not, align them:
r2 <- resample(r2, r1)

# layer names
names(cube) <- c("layer1", "layer2")

# time slices
time(cube) <- as.Date(c("2024-01-01", "2024-02-01"))

st_dimensions(cube)$band <- c("layer1", "layer2")
st_dimensions(cube)$time <- as.Date(c("2024-01-01", "2024-02-01"))
```

Practice Questions

1. You are given a large land cover raster and a polygon shapefile representing watershed boundaries. How would you extract raster values for a single watershed efficiently?
 1. What if the raster and vector layers are in different projections?
 2. How would you handle `nodata` values?
2. You need to compute pixel-wise subtraction between two rasters with different resolutions and extents. How would you prepare them?
3. How would you calculate mean elevation per administrative boundary polygon from a DEM?
4. Describe how you would convert building footprint polygons into a raster grid with building counts per cell.
5. You are working with a 200GB raster dataset. How would you process it without loading it entirely into memory?
6. In a socio-economic vector data cube with variables population, life expectancy, and gross domestic product ordered by dimensions country and year, which variables have block support for the spatial dimension, and which have block support for the temporal dimension?